

Lecture 2: Hadoop Distributed File System (HDFS)

DSL351 / DSP352

Amit Kumar Dhar

ED1, 406A

amitkdhar@iitbhlai

January 7, 2026

Introduction to HDFS

The Storage Challenge

Scenario: You have a 10 TB dataset (e.g., log files from a popular website).

Problem

- Standard hard drives are 2TB - 4TB.
- Reading 10 TB from a single disk @ 100 MB/s takes $\approx$ 28 hours.
- What if the disk fails? Data is lost.

Solution: Distributed Storage (HDFS).

What is HDFS?

Hadoop Distributed File System (HDFS) is a distributed file system designed to run on commodity hardware.

- **Highly Fault-Tolerant:** Assumes hardware failure is the norm, not the exception.
- **High Throughput:** Optimized for batch processing (write once, read many).
- **Large Datasets:** Works best with large files (GBs to TBs).

Analogy: Instead of one giant super-computer, use a army of cheap laptops.

HDFS Architecture

Master / Slave Architecture

HDFS uses a strict Master/Slave architecture.

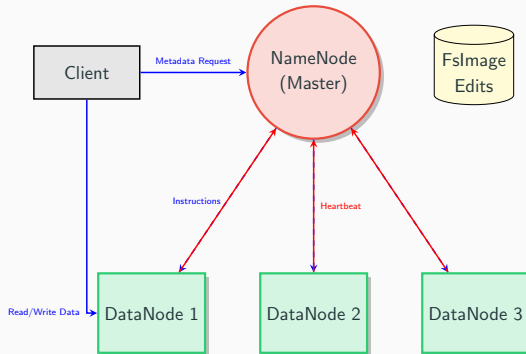
NameNode (Master)

- The "Traffic Controller".
- Stores **Metadata** (file names, permissions, hierarchy).
- Maintains the mapping of file → blocks → DataNodes.
- **Single Point of Failure** (in Hadoop 1.x).

DataNode (Slave)

- The "Workhorse".
- Stores actual data blocks.
- Handles read/write requests from clients.
- Sends **Heartbeats** and **Block Reports** to NameNode.

HDFS Architecture Diagram



Blocks and Replication

The Block Concept

- Files in HDFS are split into fixed-size chunks called **Blocks**.
- **Default Size:** 128 MB (Hadoop 2.x+) or 64 MB (Hadoop 1.x).

Why 128 MB?

To minimize the cost of seeks. If block is too small, we spend too much time finding the start of the block (seek time) vs. reading the data (transfer rate).

Goal: Time to seek $\approx$ 1% of Time to transfer.

Replication Strategy

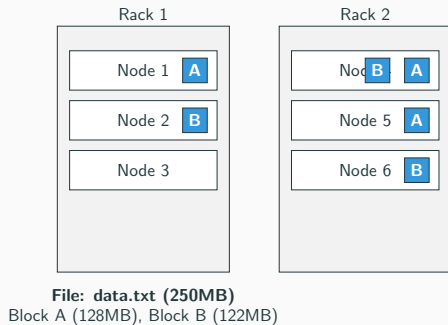
To handle hardware failure, every block is replicated.

- **Default Replication Factor:** 3

Placement Policy (Rack Awareness):

1. **Replica 1:** On the local node (where the writer is) or a random node.
2. **Replica 2:** On a different rack (to survive switch/rack failure).
3. **Replica 3:** On a different node in the *same* rack as Replica 2.

Visualizing Replication (Factor = 3)



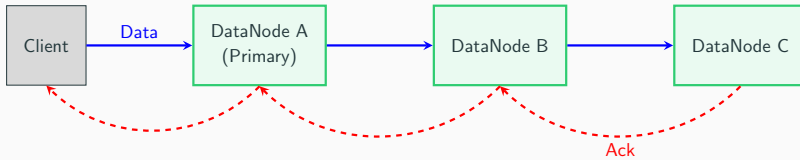
Read and Write Paths

HDFS Write Path (The Pipeline)

How does a client write a file?

1. **Client** asks NameNode to create a file.
2. **NameNode** checks permissions and returns a list of DataNodes (e.g., A, B, C) to store the first block.
3. **Client** connects to DataNode A and starts streaming data.
4. **Pipeline:** DataNode A stores the block and forwards it to B. B stores it and forwards to C.
5. **Ack:** C acks to B, B acks to A, A acks to Client.

Write Pipeline Illustration



HDFS Read Path

How does a client read a file?

1. **Client** asks NameNode for location of "file.txt".
2. **NameNode** returns list of blocks and their locations (sorted by proximity to client).
3. **Client** connects directly to the closest DataNode for Block 1.
4. **Client** reads Block 1.
5. **Client** connects to closest DataNode for Block 2, and so on.

Note: The NameNode is NOT in the data transfer path. This prevents it from becoming a bottleneck.

Reliability and High Availability

How does HDFS know if a DataNode is dead?

- **Heartbeats:** DataNodes send a signal to NameNode every 3 seconds.
- **Dead Interval:** If no heartbeat for 10 minutes (default), NameNode considers the node dead.
- **Re-replication:** NameNode notices block replication fell below factor (e.g., $3 \rightarrow 2$) and initiates replication to a new node.

HDFS High Availability (HA)

In Hadoop 1.x, the NameNode was a Single Point of Failure (SPOF).

Hadoop 2.x+ HA Architecture:

- Two NameNodes: **Active** and **Standby**.
- **Shared Edits Log:** Both nodes have access to a shared storage (JournalNodes) to keep metadata in sync.
- **Fast Failover:** If Active dies, Standby takes over immediately using ZooKeeper.

Problem: The NameNode keeps all metadata in RAM. What if we have too many files for one server's RAM?

Solution: Federation

- Use multiple NameNodes.
- Each NameNode manages a specific **Namespace Volume** (e.g., NN1 manages '/user', NN2 manages '/tmp').
- All NameNodes share the same pool of DataNodes.

Practical HDFS Usage

1. Interacting via CLI

The primary way to interact with HDFS is the command line interface.

```
# List files in root directory
```

```
hdfs dfs -ls /
```

```
# Create a directory
```

```
hdfs dfs -mkdir /user/student/data
```

```
# Upload a local file to HDFS (Write)
```

```
hdfs dfs -put local_file.txt /user/student/data/
```

```
# Download a file from HDFS to local (Read)
```

```
hdfs dfs -get /user/student/data/local_file.txt .
```

```
# Read file content directly to stdout
```

```
hdfs dfs -cat /user/student/data/local_file.txt
```

2. Python: Reading/Writing (PyArrow)

Using 'pyarrow' to interact with HDFS programmatically.

```
import pyarrow.fs as fs
```

```
# Connect to HDFS (assuming default port 8020)
```

```
hdfs = fs.HadoopFileSystem("namenode", 8020)
```

```
# Write Data
```

```
with hdfs.open_output_stream("/user/data/test.txt") as stream:  
    stream.write(b"Hello-HDFS-from-Python!")
```

```
# Read Data
```

```
with hdfs.open_input_stream("/user/data/test.txt") as stream:  
    data = stream.read().decode("utf-8")  
    print(data)
```

3. Scala: Reading Files (Hadoop API)

Using the native Java/Scala Hadoop API.

```
import org.apache.hadoop.conf.Configuration
import org.apache.hadoop.fs.{FileSystem, Path}
import java.io.{BufferedReader, InputStreamReader}

val conf = new Configuration()
val fs = FileSystem.get(conf)
val path = new Path("/user/data/test.txt")

// Open and Read
val stream = fs.open(path)
val reader = new BufferedReader(new InputStreamReader(stream))
println(reader.readLine())
reader.close()
```

4. Scala: Writing Files (Hadoop API)

```
import org.apache.hadoop.fs.{FileSystem, Path}
import java.io.BufferedWriter
import java.io.OutputStreamWriter

val path = new Path("/user/data/output.txt")

// Create and Write (Overwrite = true)
val outStream = fs.create(path, true)
val writer = new BufferedWriter(new OutputStreamWriter(outStream))

writer.write("Writing-to-HDFS-from-Scala-is-easy!")
writer.newLine()

writer.close()
fs.close()
```


Summary

Key Takeaways

1. **Distributed Storage:** HDFS splits large files into 128MB blocks.
2. **Reliability:** Blocks are replicated (usually 3x) across different nodes/racks.
3. **Scalability:** Add more DataNodes to increase storage. Use Federation to scale metadata.
4. **Architecture:** Master (NameNode) manages metadata; Slaves (DataNodes) store data.
5. **Access:** Write-once, Read-many model optimized for high throughput.

Questions?